

Aplicación interactiva para simulación de enfermedades contagiosas en entornos académicos

Cristian Rey Márquez



1 RESUMEN DEL REPORTE

ESTE informe recoge la evolución del proyecto desde la entrega del Informe Inicial, con una actualización del proyecto y los objetivos. El objetivo general del TFG se mantiene: desarrollar una aplicación interactiva que permita construir un escenario académico personalizado y ejecutar simulaciones en masa para analizar la propagación y el impacto de la organización académica (especialmente horarios).

En el tiempo transcurrido se ha trabajado principalmente en el desarrollo del constructor de escenarios, implementando el modelo de datos de la facultad, el funcionamiento de las funciones principales de construcción en el *backend* y un controlador para coordinar la interacción de los usuarios con la interfaz de la aplicación y la lógica del código.

Actualmente se está trabajando en la interfaz interactiva del usuario y en la conexión de todas las funciones del código con esta. La parte de la simulación SEIR, ejecución en *batch* y visualización y analítica de los datos está aun pendiente, tal y como estaba planificado.

2 ACTUALIZACIÓN DE LOS OBJETIVOS DEL TFG

2.1 Objetivo general

El objetivo principal se mantiene sin cambios.

2.2 Objetivos específicos

A continuación, hablamos del estado de los objetivos específicos:

- **Definir el modelo de datos del escenario académico**, incluyendo los elementos mínimos necesarios. → Estado: **Completado**
- **Implementar un flujo guiado de creación de escenarios** dentro de la aplicación, que permita introducir y editar la información básica de la facultad de forma íntegra y consistente. → Estado: **En Progreso**, *Builder* UI en desarrollo e integración del UI con el *backend* en curso.
- **Diseñar e implementar un motor de simulación basado en agentes** que modele la dinámica de contagio. → Estado: **Pendiente**, Se iniciará tras la integración del *builder*.
- **Desarrollar un módulo de análisis y**

visualización que permita al usuario interpretar los resultados de forma clara. → Estado: **Pendiente**, Se iniciará cuando se comiencen a tener simulaciones estables.

- Habilitar la visualización de simulaciones independientes de forma interactiva.
 - Habilitar la ejecución masiva de simulaciones sobre un mismo escenario.
 - Extraer y almacenar métricas relevantes de cada simulación y del conjunto de simulaciones.
- **Estudiar la seguridad de los horarios de la Escola d'Enginyeria de la UAB** en base a resultados extraídos de la aplicación. → Estado: **Pendiente**, Dependiente de la simulación y la exportación de datos.

3 PROGRESO REALIZADO DESDE EL INFORME ANTERIOR

3.1 Implementado

- **Modelo de datos de la facultad**: estructuras principales del escenario académico implementadas para representar la información base necesaria (clases/carreras/horarios, etc.).
- **Funciones de modificación del escenario**: funcionalidades del código operativas para dar soporte a la creación y gestión del escenario.
- **Controlador del backend**: implementación de una API que coordina las funciones de gestión de escenarios y de una clase que realiza las llamadas a esta API a partir de los eventos recogidos por la interfaz de usuario.
- **Decisión de diseño de la UI**: definición de como debe verse y comportarse la UI del constructor.

3.2 En progreso

- **UI interactiva del builder (PySide6)**: implementación de la interfaz y comportamiento interactivo como selección o edición.
- **Conexión frontend-backend**: integración del *builder* con el controlador para que las acciones en UI modifiquen el escenario y se reflejen correctamente.
- **Sistema de logs y ajustes**: diseño e

implementación de un sistema básico para registrar eventos y gestionar configuraciones y ajustes de la aplicación.

4 DECISIONES DE DISEÑO

4.1 Visión general

La aplicación se ha diseñado con una separación explícita entre dos conjuntos de clases que evolucionan de forma independiente:

1. **Clases de dominio (*backend*):** la clase *Facultad* y sus entidades asociadas, responsables de representar el escenario académico y toda la lógica relacionada con nodos, relaciones y validaciones del modelo.
2. **Clases de interfaz (*frontend*):** la ventana principal de la UI y sus componentes (paneles, widgets, acciones de usuario), responsables de la interacción, visualización y navegación.

Para coordinar ambos mundos sin acoplarlos directamente, se introduce una tercera pieza: el *Controller*, que actúa como clase comunicadora entre el dominio y la interfaz.

4.2 Árbol de dominio

El *backend* se estructura alrededor de una clase *Facultad*, que funciona como “fuente de verdad” del escenario, el cual se modela como un grafo donde cada sala está representada por un nodo. Su responsabilidad es:

- Mantener y almacenar el estado del escenario.
- Ofrecer operaciones para crear, editar y eliminar objetos.
- Validar la consistencia de la estructura del escenario.
- Proporcionar los datos necesarios para que la UI pueda representarlos.

4.3 Árbol de interfaz

La UI se organiza en torno a una ventana principal que coordina los distintos componentes (paneles, vistas y formularios). Sus responsabilidades principales son:

- Capturar las acciones del usuario.
- Mostrar el estado actual del escenario.
- Ofrecer un flujo guiado de edición en el *builder*.

Este árbol se diseña como una capa visual y de interacción: no implementa reglas del modelo ni “decide” la lógica del escenario, sino que delega esas operaciones al *Controller*.

4.4 Controller como mediador/comunicador

El *Controller* es la pieza que conecta ambos árboles de forma controlada. Su papel es:

- Traducir eventos de la UI en acciones sobre el escenario.
- Gestionar el estado de la interacción.
- Solicitar al modelo los datos necesarios para refrescar la vista en la interfaz.
- Notificar a la UI cuando debe actualizarse.

5 SEGUIMIENTO DE PLANIFICACIÓN Y AJUSTES

En el informe inicial se estableció que, para el Informe de Progreso I, el objetivo era tener el modelado del escenario académico terminado y quedar listo para empezar la simulación.

5.1 Estado actual respecto a este hito:

- El modelo de datos y *backend* están implementados (alineado con el hito).
- La parte que condiciona el “listo para simular” es la finalización del *builder* y, por lo tanto, la integración de la interfaz con el *backend* (actualmente en progreso).

5.2 Próximos pasos hasta Progreso II

1. Completar la integración de la interfaz, el controlador y el modelo.
2. Implementar la generación de red de contactos.
3. Implementar motor SEIR y transmisión sobre la red.
4. Implementar *batch*, reproducibilidad y almacenamiento de resultados.
5. Implementar visualizaciones mínimas:
 - a. Curvas S/E/I/R.
 - b. Infecciones por sala.
 - c. Infecciones por carrera.

6 BIBLIOGRAFIA

- [1] H. W. Hethcote, “The Mathematics of Infectious Diseases,” *SIAM Review*, vol. 42, no. 4, pp. 599–653, 2000, doi: 10.1137/S0036144500371907.
- [2] C. C. Kerr et al., “Covasim: An agent-based model of COVID-19 dynamics and interventions,” *PLoS Computational Biology*, vol. 17, no. 7, p. e1009149, jul. 2021, doi: 10.1371/journal.pcbi.1009149.
- [3] A. Colubri, M. Kemball, K. Sani, C. Boehm, K. Mutch-Jones, B. Fry, T. Brown, and P. C. Sabeti, “Preventing Outbreaks through Interactive, Experiential Real-Life Simulations,” *Cell*, vol. 182, no. 6, pp. 1366–1371, Sep. 2020, doi: 10.1016/j.cell.2020.08.042.
- [4] Operation Outbreak, “Resources,” Operation Outbreak. [Online]. Available: <https://operationoutbreak.org/resources/>. Accessed: Mar. 6, 2026.

APÉNDICE

A1. Diagrama de Gannt

